SafeCity Platform – Application Architecture

Enterprise Architecture Document | Cognizant

1. Introduction

SafeCity is a modular smart-city platform implemented using a microservices-based architecture. The platform supports identity management, incident handling, patrol operations, crisis response, and emergency dispatch capabilities.

2. Architectural Style

The solution adopts a layered microservices architecture aligned with Cognizant enterprise and cloud-native design principles.

- Presentation Layer (Web / Mobile)
- API Gateway Layer
- Microservices Layer
- Data Persistence Layer
- Integration Layer

3. Microservices & Responsibilities

IAM (Identity and Access Management)

- Register user
- Login user
- Delete user
- Update user
- Change password
- Forgot password
- Internal user lookup by ID

IRCM (Incident & Case Management)

- Incident create
- Pending incident list
- All incidents list
- Incident status update
- Case create
- Case close / resolve
- Incident retrieval views

PFOM (Patrol & Field Operations Management)

- Patrol schedule
- My patrols
- Patrol status update
- Patrol history
- Field report submit
- All field reports
- Field report review

- My field report history

DCR (Deployment & Crisis Response)

- Team create
- All teams list
- Active teams list
- Team status update
- Crisis create
- Crisis by ID
- All crises list
- Active crises list
- Crisis update
- Deploy team
- Reassign team
- Cancel deployment
- Active deployments list
- Response progress update
- Mission close

EDRA (Emergency Dispatch & Resource Allocation)

- Resource add
- All resources list
- Resource assign to incident
- Dispatch status update
- Dispatch complete
- Dispatcher dashboard view

4. Data Architecture

Each microservice maintains an independent local database or persistence store to ensure loose coupling and autonomous scalability.

5. Integration & Communication

Synchronous communication between services is handled via REST APIs using HttpClient. The platform avoids shared domain libraries to preserve service autonomy.

6. Security Architecture

Security is centralized through the IAM service, providing authentication, authorization, and access control mechanisms.

7. Non■Functional Requirements

- Scalability through independent service deployment
- High availability using stateless services
- Maintainability via clear service boundaries

8. Assumptions & Constraints

- SafeCity.Domain removed

- Each service uses local DB/library
- Cross-service data handled via HttpClient